

Formal Methods with Examples

Chandan Karfa

IIT Guwahati



ভাৰতীয় প্ৰযুক্তিবিদ্যা প্ৰতিষ্ঠান গুৱাহাটী
भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

- 1 Formal Verification Methods
- 2 Verifying a Modulo 3 Counter Design
 - Model Checking
 - Theorem Proving
- 3 Equivalence Checking between two Combinational Circuits
 - Equivalence Checking

- 1 **Formal Verification Methods**
- 2 **Verifying a Modulo 3 Counter Design**
 - Model Checking
 - Theorem Proving
- 3 **Equivalence Checking between two Combinational Circuits**
 - Equivalence Checking

Formal Verification Methods

- Model Checking
- Theorem Proving
- Equivalence Checking

Model Checking

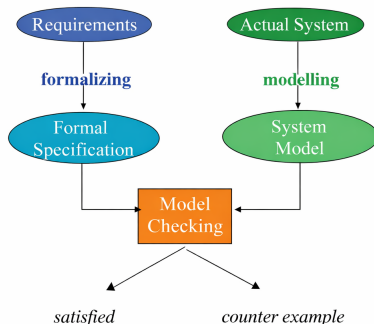


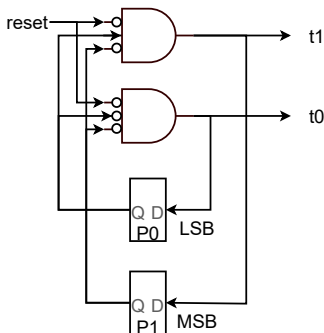
Figure 1: Model Checking

- 1 Formal Verification Methods
- 2 **Verifying a Modulo 3 Counter Design**
 - Model Checking
 - Theorem Proving
- 3 Equivalence Checking between two Combinational Circuits
 - Equivalence Checking

Model Checking Approach

Given design

A Modulo 3 Counter Design with *active high* reset



(R)	PS	NS
1	XX	00
0	00	01
0	01	10
0	10	00
0	11	00

Figure 3: State transition table for modulo-3 counter with active-high reset

Figure 2: Example design

Model Checking Approach contd.

Designing the FSM

Assume that the initial states of the Flip-Flops are 0

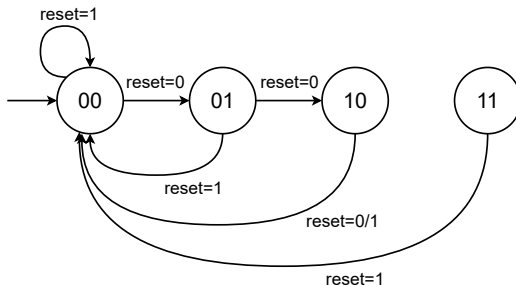


Figure 4: FSM of the design

Verification Approach

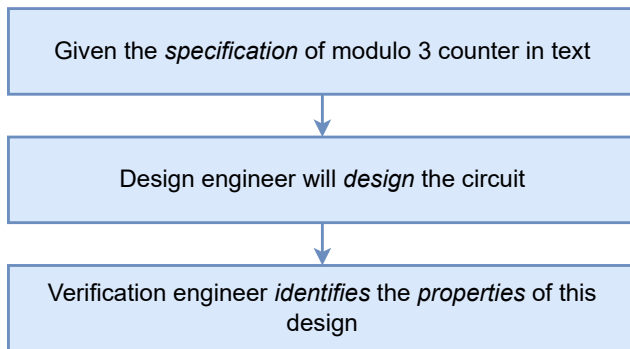


Figure 5: Standard verification approach

Properties of a modulo 3 counter

P1: The Reset Property

Irrespective of the current state, if $reset = 0$, the next state will be 00

$$G(reset = 1 \implies X(P0 = 0 \wedge P1 = 0))$$

Properties of a modulo 3 counter contd.

P2: Correct State Sequence Property

The correct state sequence must be followed in any execution of the counter ($00 \implies 01 \implies 10 \implies 00$)

$$\begin{aligned} G &(((\text{reset} = 0) \wedge (P1 = 0 \wedge P0 = 0) \implies X(P1 = 0 \wedge P0 = 1)) \wedge \\ &((\text{reset} = 0) \wedge (P1 = 0 \wedge P0 = 1) \implies X(P1 = 1 \wedge P0 = 0)) \wedge \\ &((\text{reset} = 0) \wedge (P1 = 1 \wedge P0 = 0) \implies X(P1 = 0 \wedge P0 = 0))) \end{aligned}$$

Properties of a modulo 3 counter contd.

P3: Valid State Property

At any time, the counter must be in one of the valid states 00, 01, 10

$$G(\neg(P1 = 1 \wedge P0 = 1))$$

Model Checking Approach

- From the design identify the *state machine* (the model)
- Check if the *properties* (P1, P2, P3) hold in the *state machine*

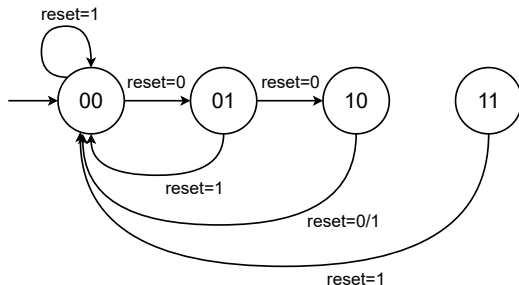
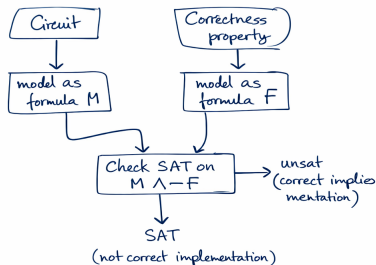


Figure 6: FSM of the design

Theorem Proving Approach

- Encode the circuit behaviour in formula(e) using *PL*, *FOL*, *FOL(T)* or *HOL* (depending on the circuit)
- Define the *next state* in terms of the *current state* and *input*



Theorem Proving Approach contd.

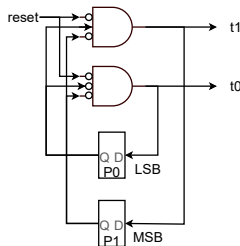
Nomenclature

Present State : $P1, P0$ Next State : $P1_ns, P2_ns$

Design Modelling

$$F1 : P1_ns = (\neg reset \wedge (P0 \wedge \neg P1))$$

$$F2 : P0_ns = (\neg reset \wedge (\neg P0 \wedge \neg P1))$$



Theorem Proving Approach contd.

Property Modelling

Property1 : reset $\implies (P1_ns == 0 \wedge P0_ns == 0)$

Theorem Proving Approach contd.

Properties contd.

Property2 : $((P1 = 0 \wedge P0 = 0) \implies P1_ns = 0 \wedge P0_ns = 1) \wedge$

$((P1 = 0 \wedge P0 = 0) \implies P1_ns = 0 \wedge P0_ns = 1) \wedge$

$((P1 = 0 \wedge P0 = 0) \implies P1_ns = 0 \wedge P0_ns = 1))$

Property3 : $\neg(P1_ns = 1 \wedge P0_ns = 1)$

Theorem Proving Approach

Combined Formula

$$F : ((F1 \wedge F2) \wedge \neg (Property1 \wedge Property2 \wedge Property3))$$

Verification

$$Check_SAT(F)$$

Steps for Theorem Proving

- Model the *design* as *formulae*
- Model the *properties* as *formulae*
- Check *satisfiability* of the *conjunction* of the formulae

- 1 Formal Verification Methods
- 2 Verifying a Modulo 3 Counter Design
 - Model Checking
 - Theorem Proving
- 3 Equivalence Checking between two Combinational Circuits
 - Equivalence Checking

Equivalence Checking Approach

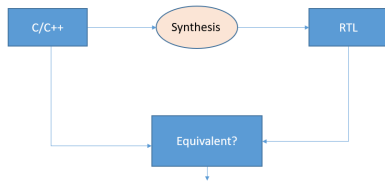


Figure 9: Equivalence Checking

Equivalence Checking Approach

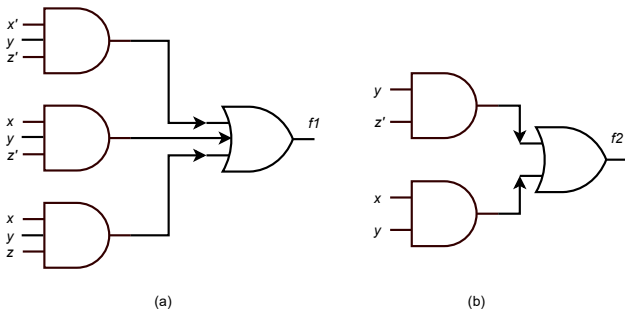


Figure 10: Given circuits implementing $f1$ and $f2$

Equivalence Checking Approach

Given

Two circuits implementing functions:

$$f_1(x, y, z) = x'yz' + xyz' + xyz$$

$$f_2(x, y, z) = yz' + xy$$

Verification task

$$Is\ f1 \equiv f2?$$

Verification Approach

Step1

Encode the design as formulae

$$f1 \text{ and } f2$$

(Already done in previous slide)

Step 2

Prove that

$$f1 \equiv f2$$

How to prove $f1 \equiv f2$?

Brute force approach

Truth table comparison of both $f1$ and $f2$.

- too many variables - size of table exponential in the number of variables

Using SAT solvers

Map this equivalence checking problem into a satisfiability problem and use SAT solver to prove equivalence.

- Can we check $f1 \equiv f2$ using a SAT solver?

How to prove $f1 \equiv f2$?

Using SAT solvers

Map this equivalence checking problem into a satisfiability problem and use SAT solver to prove equivalence.

- Can we check $f1 \equiv f2$ using a SAT solver? No, since this will give us only one assignment for which $f1 = f2$
 $\implies$ Insufficient
- We should check $f1 \neq f2$
 - It will try to find a value of the input for which $f1$ and $f2$ has a different output
 - If it returns UNSAT, it means no such input was found
 $\implies$ for every input they have the same output
 $\implies$ they are equivalent

Equivalence of Sequential Circuits

How to check for equivalence between two *sequential* circuits?

- We have to check the equivalence expression of each register/flip-flop.
- Correspondence between the registers is needed.

Model Checking Approach

Given design

A Modulo 3 Counter Design with *active high* reset

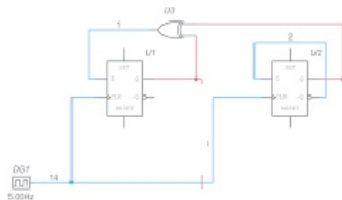
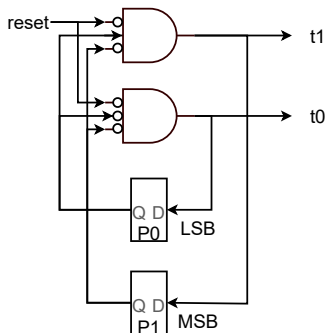


Figure 11: Example design 1

Figure 12:

Thank You